



THỰC HÀNH KỸ THUẬT LẬP TRÌNH

Tóm tắt

Tài liệu hướng dẫn thực hành Kỹ thuật lập trình củng cố cho sinh viên phương pháp lập trình có cấu trúc trên ngôn ngữ lập trình C. Vận dụng được các kiểu dữ liệu mảng một chiều, mảng hai chiều, mảng các cấu trúc, con trỏ, tập tin trong lưu trữ và xử lý dữ liệu cho các bài toán thực tế.

KHOA CÔNG NGHỆ THÔNG TIN

Biên soạn: TS. Văn Thị Thiên Trang

Lưu hành nội bộ, 2022

MỤC LỤC

BÀI THỰC HÀNH 1: MẢNG MỘT CHIỀU	1
1.1 TỔNG QUAN	1
1.1.1 Giới thiệu	1
1.1.2 Mục tiêu	1
1.1.3 Chuẩn đầu ra	1
1.2 NỘI DUNG	1
1.2.1 Tóm tắt kiến thức	1
1.2.2 Thực hành cơ bản	3
1.3 TỔNG KẾT	8
BÀI THỰC HÀNH 2: MẢNG 1 CHIỀU (TIẾP THEO) & CHUỖI KÝ TỰ	10
2.1 TỔNG QUAN	10
2.1.1 Giới thiệu	10
2.1.2 Mục tiêu	10
2.1.3 Chuẩn đầu ra	10
2.2 NỘI DUNG	10
2.2.1 Tóm tắt kiến thức	10
2.2.2 Thực hành cơ bản	12
2.3 TỔNG KẾT	17
BÀI THỰC HÀNH 3: MẢNG HAI CHIỀU	18
3.1 TỔNG QUAN	18
3.1.1 Giới thiệu	18
3.1.2 Mục tiêu	18
3.1.3 Chuẩn đầu ra	18
3.2 NỘI DUNG	18
3.2.1 Tóm tắt kiến thức	18
3.2.2 Thực hành cơ bản	19
3.3 TỔNG KẾT	21
BÀI THỰC HÀNH 4: KIỂU DỮ LIỆU CÓ CẤU TRÚC	22
4.1 TỔNG QUAN	22
4.1.1 Giới thiệu	22
4.1.2 Mục tiêu	22
4.1.3 Chuẩn đầu ra	22
4.2 NỘI DUNG	22
4.2.1 Tóm tắt kiến thức	22
4.2.2 Thực hành cơ bản	24
4.3 TỔNG KẾT	27
BÀI THỰC HÀNH 5: KIỂU DỮ LIỆU CÓ CẤU TRÚC (TIẾP THEO)	28
5.1 TỔNG QUAN	28
5.1.1 Giới thiệu	28

5.1.2 Mục tiêu.....	28
5.1.3 Chuẩn đầu ra.....	28
5.2 NỘI DUNG	28
5.2.1 Tóm tắt kiến thức.....	28
5.2.2 Thực hành cơ bản.....	29
5.3 TỔNG KẾT	34
BÀI THỰC HÀNH 6: KIỂU DỮ LIỆU CÓ CẤU TRÚC (TIẾP THEO).....	35
6.1 TỔNG QUAN	35
6.1.1 Giới thiệu.....	35
6.1.2 Mục tiêu.....	35
6.1.3 Chuẩn đầu ra.....	35
6.2 NỘI DUNG	35
6.2.1 Thực hành cơ bản.....	35
6.3 TỔNG KẾT	36
BÀI THỰC HÀNH 7: KIỂU CON TRỎ	38
7.1 TỔNG QUAN	38
7.1.1 Giới thiệu.....	38
7.1.2 Mục tiêu.....	38
7.1.3 Chuẩn đầu ra.....	38
7.2 NỘI DUNG	38
7.2.1 Tóm tắt kiến thức.....	38
7.2.2 Thực hành cơ bản.....	39
7.3 TỔNG KẾT	41
BÀI THỰC HÀNH 8: ĐỆ QUY	42
8.1 TỔNG QUAN	42
8.1.1 Giới thiệu.....	42
8.1.2 Mục tiêu.....	42
8.1.3 Chuẩn đầu ra.....	42
8.2 NỘI DUNG	42
8.2.1 Tóm tắt kiến thức.....	42
8.2.2 Thực hành cơ bản.....	43
8.3 TỔNG KẾT	43
BÀI THỰC HÀNH 9: TẬP TIN.....	45
9.1 TỔNG QUAN	45
9.1.1 Giới thiệu.....	45
9.1.2 Mục tiêu.....	45
9.1.3 Chuẩn đầu ra.....	45
9.2 NỘI DUNG	45
9.2.1 Tóm tắt kiến thức.....	45
9.2.2 Thực hành cơ bản.....	47
9.3 TỔNG KẾT	50

Bài thực hành 1: MẢNG MỘT CHIỀU

1.1 TỔNG QUAN

1.1.1 Giới thiệu

- Bài thực hành số 1: luyện tập sử dụng kiểu dữ liệu danh sách - Mảng một chiều.
- Thời gian thực hành: trên lớp 3 tiết, tự học 3 tiết.
- Yêu cầu chuẩn bị: sinh viên cài đặt phần mềm trên máy tính cá nhân (ở nhà), đã nắm lý thuyết về kiểu dữ liệu mảng, các thao tác cơ bản trên mảng một chiều.

1.1.2 Mục tiêu

Sau khi thực hành xong bài này, sinh viên có thể nắm được:

- Cách khai báo dữ liệu kiểu mảng.
- Các thao tác nhập, xuất mảng cơ bản và nhập, xuất mảng có điều kiện.
- Các kỹ thuật tính toán, tìm kiếm.

1.1.3 Chuẩn đầu ra

- Hiểu được phương pháp lập trình có cấu trúc, cách phân chia chương trình lớn thành các hàm
- Vận dụng được cấu trúc dữ liệu mảng một chiều trong lưu trữ và xử lý dữ liệu.

1.2 NỘI DUNG

1.2.1 Tóm tắt kiến thức

a) Nhắc lại cấu trúc chương trình C:

Cách 1:

Khai báo thư viện
Hằng số

Khai báo các hàm con
(nguyên mẫu hàm);

```
Hàm main()
{
    //gọi hàm con ra
    thực hiện
}
```

Các hàm con

Cách 2:

Khai báo thư viện
Hằng số

Các hàm con

```
Hàm main()
{
    //gọi hàm con ra
    thực hiện
}
```

- Có 2 loại hàm:
 - + Hàm không trả về trị. Ví dụ: Nhập/Xuất
 - + Hàm trả về trị: Sau khi thực hiện xong, giá trị kết quả được lưu lại để dùng về sau. Ví dụ: Tìm, Tính, Đếm, Kiểm tra...
- Mẫu hàm:

Hàm không trả về trị:

```
void Ten_Ham(khai báo biến tham số
nếu có)
{
    Các câu lệnh;
}
```

Hàm trả về trị:

```
Kieu_dl Ten_Ham(khai báo biến tham số
nếu có)
{
    Các câu lệnh;
    Return ket_qua;
}
//kiểu dữ liệu của hàm là kiểu của giá trị kết
quả trả về.
```

b) Kiểu dữ liệu mảng một chiều

Khi cần lưu một danh sách/dãy/tập hợp các phần tử có cùng kiểu dữ liệu ta dùng Mảng một chiều.

Mảng thực chất là một biến được cấp phát bộ nhớ liên tục và bao gồm nhiều biến thành phần. Các thành phần của mảng là các biến có cùng kiểu dữ liệu và cùng tên. Do đó để truy xuất các biến thành phần, ta dùng cơ chế chỉ mục (thứ tự tính từ 0).

Khai báo mảng:

Kiểu_dữ_liệu Tên_mảng[Kích_thước];

Truy cập từng phần tử trong mảng:

Tên_biến_mảng[chỉ_số];

Để lưu trữ một danh sách, ta cần 2 biến quản lý:

- (1) Biến lưu số lượng phần tử của danh sách, ví dụ: `int n`;
- (2) Biến lưu giá trị/thông tin các phần tử trong danh sách, ví dụ: `int a[100]`;

Do đó, khi thao tác trên danh sách, dữ liệu đầu vào của hàm thực hiện thao tác phải có 2 biến này.

1.2.2 Thực hành cơ bản

Bài 1: Viết chương trình thực hiện:

1. Nhập vào một dãy số nguyên gồm n phần tử ($0 < n < 100$).
2. Xuất dãy vừa nhập.
3. Xuất các phần tử chia hết cho 3 có trong dãy số.
4. Xuất các phần tử âm chẵn có trong dãy số.
5. Xuất các phần tử nguyên tố có trong dãy số.
6. Đếm số lượng phần tử âm, chẵn, không chia hết cho 5 có trong dãy số.
7. Đếm số lượng số nguyên tố có trong dãy số.
8. Tính tổng các phần tử của dãy số.
9. Tính tổng các số nguyên tố có trong dãy số.
10. Tính trung bình cộng các phần tử của dãy số.
11. Tính trung bình cộng các số nguyên tố có trong dãy số.

12. Tìm phần tử dương đầu tiên, hãy cho biết vị trí và giá trị của phần tử tìm được (nếu có).
13. Tìm phần tử dương cuối cùng, hãy cho biết vị trí và giá trị của phần tử tìm được (nếu có).
14. Tìm phần tử có giá trị là X (với X do người dùng nhập)
15. Tìm giá trị phần tử lớn nhất (nhỏ nhất).
16. Đếm số lượng phần tử có giá trị lớn nhất.
17. Xuất ra vị trí của các phần tử có giá trị lớn nhất.

Hướng dẫn:

- Câu 1, 2: Thực hiện nhập, xuất dữ liệu. Sau khi chạy thử hai chức năng này thành công, ta mới làm các câu tiếp theo.
- (1) Nhập mảng số nguyên gồm n phần tử ($0 < n < 100$).
 - (2) Xuất mảng vừa nhập.

```
#include<stdio.h>
#define MAX 100 //hằng số qui định kích thước cấp phát cho mảng

//Định nghĩa các hàm con, mỗi hàm đảm nhiệm một công việc trong bài toán
//a) bước 1: nhập số lượng phần tử của dãy số lưu vào biến n, sao cho 0<n< MAX
void nhapSL(int &n){
    do{
        printf("Nhap so luong phan tu 0<n<=%d: ", MAX);
        scanf(".....");
    }while(n<=0 || n>=MAX);
}

// bước 2: nhập từng phần tử trong dãy số, tức nhập cho từng a[i], i=0, 1, 2...
void nhapDS(int a[], int n)
{
    for(int i=0; i<n; i++)
    {
        printf("a[%d]=", i);
```

```

        scanf(.....);
    }
}
//b) hàm xuất dãy số
void xuatDS(int a[], int n)
{
    printf("Day so:\n");
    for(int i=0; i<n; i++)
        printf(.....);
}
//=====
int main()
{
    int a[MAX], n;
    nhapSL(n); nhapDS(a, n);
    xuatDS(a, n);
    return 0;
}

```

- Câu 3, 4, 5: dạng thao tác xuất có điều kiện/ lọc dữ liệu theo điều kiện.

```

void xuatTheoDK(int a[], int n)
{
    printf("Cac phan tu thoa dieu kien:\n");
    for(int i=0; i<n; i++)
        if (điều kiện trên a[i])
            xuất a[i];
}
//Gọi thực hiện trong main():    xuatTheoDK(a, n);

```

(3) Xuất các phần tử chia hết cho 3 có trong mảng: **if (a[i]%3==0)**

(4) Xuất các phần tử âm chẵn có trong mảng: **if (a[i]<0 && a[i]%2!=0)**

(5) Xuất các phần tử nguyên tố có trong mảng: **if (ktnt(a[i])==1)**

Trong đó, hàm kiểm tra một số nguyên có là số nguyên tố không có dạng:

```
int ktnt(int x)
```



```
{
    //thực hiện các lệnh để kiểm tra tính nguyên tố của x
    //return 1; nếu x nguyên tố
    //return 0; nếu x không nguyên tố
}
```

- Câu 6, 7: dạng thao tác đếm các phần tử thỏa điều kiện.

```
int demTheoDK(int a[], int n)
{
    int i,dem=0;
    for(i=0; i<n; i++)
        if (điều kiện trên a[i])
            dem++; //hoặc dem=dem+1; hoặc dem+=1;
    return dem;
}
//Gọi thực hiện trong main():
printf("Số lượng phần tử thỏa điều kiện là %d", demTheoDK(a, n));
```

- (6) Đếm số lượng phần tử âm, chẵn, không chia hết cho 5 có trong dãy số.

if (a[i] âm && a[i] chẵn && a[i] không chia hết cho 5)

- (7) Đếm số lượng số nguyên tố có trong dãy số: tương tự câu (5)

- Câu 8, 9: tính tổng các phần tử của toàn dãy và tính tổng có điều kiện

```
int tongTheoDK(int a[], int n)
{
    int i,s=0;
    for(i=0; i<n; i++)
        if (điều kiện trên a[i]) //nếu tổng toàn dãy thì không có điều kiện này
            s=s+a[i];
    return s;
}
//Gọi thực hiện trong main():
printf("Tổng các phần tử thỏa điều kiện là %d", tongTheoDK(a, n));
```

- Câu 10, 11: tính trung bình cộng tất cả các phần tử/các phần tử thỏa điều kiện.

```
float tbcDS (int a[], int n)
```

```

{
    return 1.0*tongDS(a,n)/n;
}
float tbcTheoDK(int a[], int n)
{
    return 1.0*tongTheoDK(a,n)/demTheoDK(a,n);
}
//Gọi thực hiện trong main():
printf("TBC cac phan tu thoa dieu kien la %.1f",tbcTheoDK(a,n));

```

- Câu 12, 13, 14: dạng thao tác tìm kiếm trên mảng, kết quả trả về là đối tượng duy nhất tìm được (nếu có). Trong trường hợp nếu kết quả tìm là nhiều phần tử thì bài toán quy về dạng thao tác xuất có điều kiện.
Kết quả tìm kiếm có thể là vị trí/giá trị của đối tượng tìm được; tùy thuộc yêu cầu bài toán, ta lựa chọn kết quả trả về cho phù hợp.

```

//trả về vị trí của phần tử tìm được
int timTheoDK((int a[], int n, khai báo thêm biến dữ liệu cần cho việc tìm nếu có)
{
    for(int i=0; i<n; i++)
        if (a[i] thỏa điều kiện cần tìm)
            return i; //trả về vị trí của phần tử thỏa điều kiện
    return -1; //trả về -1 cho trường hợp không tìm thấy
}

```

(12) Tìm phần tử dương đầu tiên: **if (a[i] > 0)**

Gọi thực hiện trong main():

```

int vt=timDuongDauTien(a,n);
if (vt==-1) printf("Day so khong co phan tu duong);
else printf("Phan tu duong dau tien la %d tai vi tri %d", a[vt], vt);

```

(13) Tìm phần tử dương cuối cùng: Tương tự, thay đổi vòng lặp for để duyệt ngược từ cuối dãy số về đầu dãy số.

(14) Tìm phần tử có giá trị là X (với X do người dùng nhập): **if (a[i]==X)**

```

int timTheoDK((int a[], int n, int X)

```

Thực hiện nhập X trong hàm main, trước khi gọi hàm tìm.

- Câu 15, 16, 17:

(15) Tìm giá trị phần tử lớn nhất:

```
int timMax((int a[], int n)
{
    int max=a[0]; //giả sử phần tử đầu tiên có giá trị lớn nhất
    for(int i=0; i<n; i++) //duyet dãy số
        if (a[i] >max) //nếu có phần tử nào lớn hơn giá trị lớn nhất hiện có
            max=a[i]; //cập nhật lại giá trị lớn nhất
    return max;
}
```

(16) Đếm số lượng phần tử có giá trị lớn nhất: áp dụng thao tác đếm có điều kiện.

```
int demTheoDK(int a[], int n)
{
    //lấy giá trị lớn nhất bằng cách gọi hàm ở câu (15)
    int max=timMax(a,n);
    //thực hiện đếm các phần tử có giá trị = max
    ... bạn tự code
}
```

(17) Xuất ra vị trí của các phần tử có giá trị lớn nhất: áp dụng thao tác xuất có điều kiện.

```
void xuatViTriCuaMax(int a[], int n)
{
    //lấy giá trị lớn nhất bằng cách gọi hàm ở câu (15)
    int max=timMax(a,n);
    printf("Phan tu lon nhat co gia tri la %d nam o cac vi tri: ", max);
    //Duyệt dãy số, nếu phần tử a[i] nào bằng giá trị lớn nhất thì xuất vị trí i
    ... bạn tự code
}
```

Bài 2: Tổ chức lại hàm main() để chương trình thực hiện theo menu chức năng.

1.3 TỔNG KẾT

Sau bài thực hành này, người học cần hiểu và nắm vững:

- Dữ liệu kiểu mảng dùng cho việc biểu diễn những thông tin có cùng kiểu dữ liệu liên tiếp nhau (dữ liệu dạng danh sách).
- Khi cài đặt bài tập mảng một chiều nên xây dựng thành những hàm chuẩn để dùng lại cho các bài tập khác.
- Các thao tác trên mảng đều theo quy tắc nhất định, chúng ta có thể ứng dụng mảng trong việc biểu diễn số lớn, dùng bảng tra, khử đệ qui, ...

Bài thực hành 2: MẢNG 1 CHIỀU (TIẾP THEO) & CHUỖI KÝ TỰ

2.1 TỔNG QUAN

2.1.1 Giới thiệu

- Bài thực hành số 2: luyện tập các thao tác nâng cao trên mảng, thao tác với chuỗi ký tự.
- Thời gian thực hành: trên lớp 3 tiết, tự học 3 tiết.
- Yêu cầu chuẩn bị: sinh viên đã nắm lý thuyết về kiểu dữ liệu mảng, các thao tác nâng cao trên mảng một chiều, thao tác với chuỗi ký tự.

2.1.2 Mục tiêu

Sau khi thực hành xong bài này, sinh viên có thể nắm được:

- Các kỹ thuật thêm/xóa phần tử, kiểm tra mảng có thỏa điều kiện, tạo mảng mới từ mảng đã có, tách và ghép mảng...
- Khai báo và sử dụng biến chuỗi để lưu trữ dãy ký tự.
- Thao tác xử lý chuỗi ký tự, vận dụng được các hàm xử lý chuỗi có sẵn trong thư viện.

2.1.3 Chuẩn đầu ra

- Sử dụng được mảng để lưu trữ danh sách các phần tử, cài đặt được các thao tác nâng cao trên mảng.
- Hiểu được cấu trúc dữ liệu chuỗi ký tự. Xử lý và thao tác được với chuỗi ký tự.

2.2 NỘI DUNG

2.2.1 Tóm tắt kiến thức

a) Chuỗi ký tự

Chuỗi ký tự là trường hợp đặc biệt của mảng một chiều. Chuỗi ký tự là một dãy các phần tử, mỗi phần tử có kiểu ký tự.

Chuỗi ký tự được kết thúc bằng ký tự '\0'. Do đó khi khai báo độ dài của chuỗi luôn luôn khai báo dư 1 phần tử để chứa ký tự '\0'.

b) Khai báo chuỗi

Cách 1: **char Tên chuỗi [Số ký tự tối đa của chuỗi];**

Ví dụ: char s[30];

Cách 2: **char * Tên chuỗi;**

Ví dụ: char *s;

c) Nhập, xuất chuỗi**- Nhập chuỗi:**

Cách 1: **gets(biến chuỗi);**

Nhận các ký tự nhập từ phím cho đến khi nhấn phím Enter và đưa vào biến chuỗi.

Ví dụ:

```
char hoten[30];
```

```
printf("Nhap chuoi: "); fflush(stdin); gets(hoten);
```

Cách 2: **scanf("%s", &biến chuỗi);**

Nhận các ký tự nhập từ phím cho đến khi gặp phím *space*, *tab*, *new line*, *enter* thì dừng, do đó chỉ dùng hàm scanf để nhập chuỗi không có khoảng trắng.

Ví dụ:

```
char masv[10];
```

```
scanf("%s", &masv);
```

- Xuất chuỗi:

Cách 1: **puts (biến chuỗi);** //Xuất chuỗi xong tự động xuống dòng

Cách 2: **printf ("%s", biến chuỗi);**

d) Một số hàm thư viện thường dùng (thư viện <string.h>)

TT	TÊN HÀM	CHỨC NĂNG
1	int strlen(char s[]);	Trả về độ dài của chuỗi s.
2	strcpy(char des[], char src[]);	Sao chép nội dung chuỗi src vào chuỗi des.
3	strncpy(char dest[], char src[], int n);	Chép n ký tự từ chuỗi src sang chuỗi dest. Nếu chiều dài src < n thì hàm sẽ điền khoảng trắng cho đủ n ký tự vào dest.
4	strcat(char s1[],char s2[]);	Nối chuỗi s2 vào chuỗi s1.

TT	TÊN HÀM	CHỨC NĂNG
5	<code>strncat(char s1[],char s2[],int n)</code>	Nối n ký tự đầu tiên của chuỗi $s2$ vào chuỗi $s1$.
6	<code>int strcmp(char s1[],char s2[])</code>	So sánh 2 chuỗi $s1$ và $s2$ theo nguyên tắc thứ tự từ điển. Phân biệt chữ hoa và thường. Trả về: • 0 : nếu $s1$ bằng $s2$. • >0: nếu $s1$ lớn hơn $s2$. • <0: nếu $s1$ nhỏ hơn $s2$.
7	<code>int stricmp(char s1[],char s2[])</code>	Tương tự như <code>strcmp()</code> , nhưng không phân biệt hoa thường.
8	<code>char *strstr(char s1[], char s2[]);</code>	Tìm sự xuất hiện đầu tiên của chuỗi $s2$ trong chuỗi $s1$. Trả về: • NULL: nếu không có. • Ngược lại: Địa chỉ bắt đầu chuỗi $s2$ trong $s1$.
9	<code>char *strupr(char *s)</code>	Đổi chuỗi chữ thường thành chuỗi chữ hoa
10	<code>char *strlwr (char *s)</code>	Đổi chuỗi chữ hoa thành chuỗi chữ thường

2.2.2 Thực hành cơ bản

Bài 1: Viết chương trình thực hiện:

1. Sinh ngẫu nhiên một dãy số nguyên gồm n phần tử ($0 < n < 100$) sao cho các phần tử $\in (-20, 20)$.
2. Xuất dãy số.
3. Thêm một phần tử vào vị trí k trong dãy, với k do người dùng nhập.
4. Thêm một phần tử vào đầu danh sách.
5. Thêm một phần tử vào cuối danh sách.
6. Xóa phần tử tại vị trí k trong dãy, với k do người dùng nhập.
7. Tìm phần tử có giá trị X , nếu tìm thấy hãy xóa phần tử đó (giả sử các phần tử trong mảng là phân biệt).
8. *Xóa tất cả phần tử có giá trị X (mảng có thể chứa phần tử trùng nhau).

9. Cho biết mảng có sắp tăng dần không?
10. Cho biết mảng có đối xứng không?
11. Tạo mảng mới chứa các phần tử dương của mảng đã có.
12. *Sắp xếp mảng tăng dần.

Hướng dẫn:

- Câu 1:
 - + Số lượng phần tử của dãy lưu trong biến n và do người dùng nhập với yêu cầu $0 < n < 100$.
 - + Mỗi phần tử trong dãy số do chương trình tự sinh ngẫu nhiên với yêu cầu $a[i] \in (-20, 20)$.

```
#include<stdio.h>
#include<time.h>
#define MAX 100
//a) bước 1: nhập số lượng phần tử của dãy số lưu vào biến n, sao cho 0<n< MAX
void nhapSL(int &n){
    do{
        printf("Nhap so luong phan tu 0<n<=%d: ", MAX);
        scanf(".....");
    }while(.....);
}
// bước 2: sinh từng phần tử trong dãy số, sinh cho từng a[i], i=0, 1, 2...
void sinhDS(int a[], int n)
{
    srand(time(NULL));
    for(int i=0; i<n; i++)
        a[i]=rand()%40-20;
}
```

- Câu 3: Thêm một phần tử vào vị trí k trong dãy, với k do người dùng nhập

```
//k: vị trí sẽ thêm, x: phần tử cần thêm
void themTaiViTri(int a[], int &n, int k, int x){
    //b1. Dịch chuyển các phần tử từ vị trí k qua phải 1 ô bằng cách sao chép giá trị
    for(int i=... ..)
```



```

        a[i]=a[i-1];
        //b2. chèn x vào ô nhớ thứ k
        .....
        //b3. tăng số lượng phần tử lên 1
        .....
    }
    //Gọi thực hiện trong main(), thực hiện các bước sau:
    - Nhập vị trí k cần thêm sao cho  $0 \leq k \leq n$ 
    - Nhập giá trị phần tử x cần thêm vào dãy số
    - Gọi hàm thêm x vào vị trí k: themTaiViTri(a, n, k, x);
    - Gọi hàm xuất dãy số để xem dãy số sau khi thêm: xuatDS(a, n);

```

- Câu 4, câu 5:
 - + Thêm một phần tử vào đầu danh sách: `themTaiViTri(a, n, 0, x);`
 - + Thêm một phần tử vào cuối danh sách: `themTaiViTri(a, n, n, x);`
- Câu 6: Xóa phần tử tại vị trí k trong dãy, với k do người dùng nhập.

```

//k: vị trí sẽ xóa
void xoaTaiViTri(int a[], int &n, int k){
    //b1. . Dịch chuyển các phần tử từ vị trí sau k qua trái 1 ô bằng cách sao chép giá trị
    for(int i=... ..)
        a[i]=a[i+1];
    //b2. Giảm số lượng phần tử đi 1
    .....
}
//Gọi thực hiện trong main(), thực hiện các bước sau:
- Nhập vị trí k cần xóa sao cho  $0 \leq k < n$ 
- Gọi hàm xóa tại vị trí k: xoaTaiViTri(a, n, k);
- Gọi hàm xuất dãy số để xem dãy số sau khi xóa: xuatDS(a, n);

```

- Câu 7: Tìm phần tử có giá trị X, nếu tìm thấy hãy xóa phần tử đó (giả sử các phần tử trong mảng là phân biệt).
 - + Viết hàm tìm phần tử có giá trị X, trả về vị trí (chỉ số) tìm thấy X trong mảng nếu có hoặc trả về -1 nếu không tìm thấy.

```
int timX(int a[], int n, int X)
```

- + Hàm xóa phần tử tại vị trí k (đã làm ở câu 6)

```
void xoaTaiViTri(int a[], int &n, int k)
```

- + Hàm thực hiện tìm và xóa:

```
void timXoa(int a[], int n)
{
    int X;
    - nhập giá trị X cần tìm
    - int vt=timX(a,n,X);
    if (vt==-1) printf("khong tim thay %d trong mang", X);
    else
    {
        printf("Tim thay %d tai vi tri %d va day sau khi xoa la:\n", X, vt);
        xoaTaiViTri(a, n, vt);
        xuatDS(a,n);
    }
}
```

- Câu 8: bạn tự làm.
- Câu 9, 10: Kiểm tra dãy số có thỏa điều kiện nào đó không. Hàm trả về 1 nếu dãy thỏa và trả về 0 nếu không thỏa điều kiện.
 - + Mảng tăng dần nếu $a[i] < a[i+1]$ với $\forall i=0, 1, 2, \dots, n-2$
 - + Mảng đối xứng nếu $a[i] = a[n-1-i]$ với $\forall i=0, 1, 2, \dots, n-1$
- Câu 11: Tạo mảng mới chứa các phần tử dương của mảng đã có.

```
//mảng b sẽ chứa các phần tử thỏa điều kiện cần lấy ra ở mảng a
void taoMang(int a[], int n, int b[], int &z)
{
    for(int i=0; i<n; i++)
        if (a[i] thỏa điều kiện) b[z++]=a[i]; //có thể tách ra b[z]=a[i]; và z++;
}
//Gọi thực hiện trong main()
int b[MAX], z=0;
taoMang(a, n, b, z);
```

Bài 2: Viết chương trình thực hiện:

1. Nhập vào thông tin của một sinh viên gồm: mã sinh viên, họ tên, lớp.
2. Xuất các thông tin của sinh viên ra màn hình.
3. Hãy cho biết độ dài tên của sinh viên gồm bao nhiêu ký tự?
4. Hãy cho biết sinh viên này có học ngành Công nghệ thông tin hay không?
5. *Hãy chuẩn hóa họ tên của sinh viên (giả sử người dùng nhập chưa chuẩn). Ví dụ: nguyen van bao => Nguyen Van Bao
6. *Hãy đổi tên sinh viên thành viết IN HOA.

Hướng dẫn:

- Câu 1, 2: Xác định kiểu dữ liệu cần lưu trữ:
 - + Mã sinh viên: chuỗi tối đa 10 ký tự
 - + Họ tên: chuỗi tối đa 30 ký tự
 - + Lớp: chuỗi tối đa gồm 10 ký tự trong đó 2 ký tự đầu là khóa, ký tự tiếp theo là bậc học (D: đại học), 3 ký tự tiếp theo là mã ngành và 2 ký tự cuối là số thứ tự của lớp, ví dụ: 22D1ITE01.
 - + Viết hàm nhập/xuất dữ liệu cho 3 biến

//câu 1. Nhập thông tin 1 sinh viên

```
void nhapSV(char masv[], char hoten[], char lop[])
{
    printf("Mã sinh viên: ");
    scanf("%s", &masv); //nhập dãy ký tự không có khoảng trắng
    printf("Nhập họ tên: ");
    fflush(stdin);
    gets(hoten); //nhập dãy ký tự có khoảng trắng
    printf("Lop: "); scanf("%s", &lop);
}
```

//câu 2. Xuất thông tin 1 sinh viên

```
void xuatSV(char masv[], char hoten[], char lop[])
{
    printf("\nThông tin sinh viên: ");
    printf("\n%s\t%s\t%s", masv, hoten, lop);
}
```

- Câu 3: Dùng hàm *strlen(...)*
- Câu 4: Dùng hàm *strstr(...)*

2.3 TỔNG KẾT

Sau bài thực hành này, người học cần hiểu và nắm vững:

- Cũng giống như kiểu mảng một chiều, thao tác truy xuất các phần tử trên chuỗi hoàn toàn tương tự. Bên cạnh đó, kiểu dữ liệu này còn được cài đặt sẵn một số hàm thư viện rất hữu ích nên trong quá trình thao tác trên chuỗi nên khi cài đặt ta cố gắng tận dụng tối đa những hàm liên quan.
- Không nên sử dụng hàm *scanf()* để nhập chuỗi trong trường hợp chuỗi dữ liệu nhập vào có chứa khoảng trắng.
- Nếu nhập chuỗi phía sau hàm *scanf()* nên chèn hàm *fflush(stdin)* giữa *scanf()* và *gets()* để xóa vùng đệm, tránh trường hợp chương trình bỏ qua hàm *gets()* do trong vùng đệm còn lưu ký tự xuống dòng của phím ENTER.
- Khi thao tác trên chuỗi lưu ý phải đảm bảo chuỗi được kết thúc bằng ký tự kết thúc '\0'.

Bài thực hành 3: MẢNG HAI CHIỀU

3.1 TỔNG QUAN

3.1.1 Giới thiệu

- Bài thực hành số 3: luyện tập sử dụng kiểu dữ liệu Mảng hai chiều.
- Thời gian thực hành: trên lớp 3 tiết, tự học 3 tiết.
- Yêu cầu chuẩn bị: sinh viên đã nắm lý thuyết về kiểu dữ liệu mảng hai chiều, các thao tác cơ bản trên mảng hai chiều.

3.1.2 Mục tiêu

Sau khi thực hành xong bài này, sinh viên có thể nắm được:

- Cách khai báo dữ liệu kiểu mảng hai chiều.
- Các thao tác nhập, xuất mảng.
- Các kỹ thuật tính toán, xử lý trên từng dòng/cột của mảng hai chiều.

3.1.3 Chuẩn đầu ra

- Hiểu và vận dụng được cấu trúc dữ liệu dạng mảng hai chiều cho các ứng dụng phù hợp.

3.2 NỘI DUNG

3.2.1 Tóm tắt kiến thức

Mảng hai chiều thực chất là mảng một chiều trong đó mỗi phần tử của mảng là một mảng một chiều, và được truy xuất bởi hai chỉ số dòng và cột.

Mảng hai chiều còn được gọi là ma trận.

a) Khai báo

Kiểu dữ liệu Tên mảng [Số dòng tối đa][Số cột tối đa];

Ví dụ:

`int sn [8][14];` //Khai báo mảng hai chiều có tên là `sn` gồm 8 dòng, 14 cột và kiểu các phần tử là `int`.

float st [10][5]; //Khai báo mảng hai chiều có tên là *st* gồm 10 dòng, 5 cột và kiểu các phần tử là float.

b) Truy cập

Tên mảng [chỉ số dòng][chỉ số cột];

3.2.2 Thực hành cơ bản

Bài 1: Luyện tập mảng hai chiều gồm *d* dòng và *c* cột. Viết chương trình thực hiện:

1. Nhập ma trận gồm *d* dòng và *c* cột (*d*, *c* nhập từ bàn phím).
2. Xuất ma trận.
3. Xuất các phần tử âm có trong ma trận.
4. Đếm số lượng phần tử âm có trong ma trận.
5. Tính tổng các phần tử của ma trận.
6. Tính tổng các phần tử âm của ma trận.
7. Tính TBC các phần tử âm có trong ma trận.
8. Tìm giá trị phần tử lớn nhất của ma trận.
9. Xuất các phần tử nằm trên dòng *k* (*k* do người dùng nhập).
10. Tính tổng các phần tử nằm trên dòng *k*.
11. Tính tổng các phần tử trên mỗi dòng của ma trận và xuất kết quả ra màn hình.
12. *Xuất ra các dòng có tổng lớn nhất.

	0	1	2	3
0				
1				
2				

Hướng dẫn:

```
#define MAX 10
//nhập số dòng và số cột của ma trận
void NhapDC(int &d, int &c)
{
    //bạn tự code
}
//nhập giá trị cho từng phần tử của ma trận
void Nhap (int a[][MAX], int d, int c )
{
    for ( int i = 0; i < d; i ++ )
        for (int j = 0; j < c; j ++)
```

```

    {
        printf (" a[%d][%d] = ", i, j );
        scanf ("%d", &a[i][j]);
    }
}

//xuất ma trận
void Xuat (int a[][MAX], int d, int c)
{
    printf ("\nMa tran:\n");
    for (int i = 0; i < d; i++)
    {
        // Xuất các phần tử trên dòng i: a[i][0], a[i][1], a[i][2], ... , a[i][c-1],
        for (int j = 0; j < c; j++)
            printf ("%5d ", a[i][j] );
        // Xuống dòng khi xuất hết các phần tử trên 1 dòng
        printf ("\n");
    }
}

```

Nhận xét:

Để duyệt qua các phần tử của mảng hai chiều, ta duyệt theo từng dòng, với mỗi dòng xét phần tử theo từng cột. Do đó, câu lệnh để thực hiện duyệt mảng:

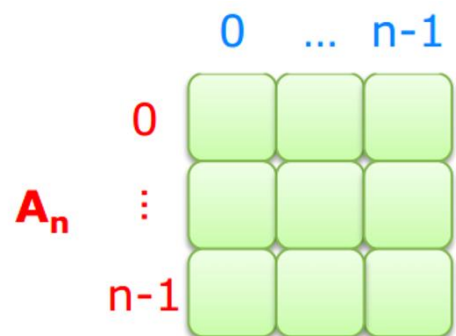
```

for ( int i = 0; i < d; i ++ )
    for (int j = 0; j < c; j ++ )
        Xét a[i][j]

```

Bài 2: Luyện tập mảng hai chiều gồm có số dòng = số cột. Ma trận vuông cấp n là mảng 2 chiều có số dòng = số cột = n .

1. Nhập ma trận vuông cấp n chứa số nguyên.
2. Xuất ma trận.
3. Liệt kê các phần tử trên đường chéo chính/ chéo phụ.
4. Tính tổng các phần tử trên đường chéo chính/ chéo phụ.
5. Tính tổng các phần tử nằm trên đường biên.



6. *Xuất các phần tử ở nửa trên đường chéo chính.
7. *Tính trung bình cộng các phần tử ở nửa dưới đường chéo phụ.
8. *Kiểm tra ma trận có đối xứng qua đường chéo chính không?

Hướng dẫn:

Vì ma trận vuông cấp n là mảng 2 chiều có số dòng = số cột = n , nên câu lệnh thực hiện duyệt các phần tử của mảng là:

```
for ( int i = 0; i < n; i ++ )
    for (int j = 0; j < n; j ++ )
        Xét a[i][j]
```

- Các phần tử nằm trên đường chéo chính có **chỉ số dòng = chỉ số cột**.
- Các phần tử nằm trên đường chéo phụ có **chỉ số dòng + chỉ số cột = $(n-1) - i$** .

	0	1	2	3	...	n-1
0	a[0][0]					
1		a[1][1]				
2			a[2][2]			
3				a[3][3]		
...					a[...][...]	
n-1						a[n-1][n-1]

Đường chéo chính

	0	1	2	3	...	n-1
0						
1						
2						
3						
...						
n-1						

Đường chéo phụ

3.3 TỔNG KẾT

Sau bài thực hành này, người học cần hiểu và nắm vững:

- Kiểu dữ liệu mảng hai chiều.
- Các thao tác xử lý trên mảng hai chiều: nhập, xuất, tính tổng, đếm, ...
- Các thao tác xử lý trên một dòng/cột, xử lý các phần tử ở các vị trí xác định.

Bài thực hành 4: KIỂU DỮ LIỆU CÓ CẤU TRÚC

4.1 TỔNG QUAN

4.1.1 Giới thiệu

- Bài thực hành số 4: làm quen với kiểu dữ liệu cấu trúc.
- Thời gian thực hành: trên lớp 3 tiết, tự học 3 tiết.
- Yêu cầu chuẩn bị: sinh viên đã nắm lý thuyết về kiểu dữ liệu struct.

4.1.2 Mục tiêu

Sau khi thực hành xong bài này, sinh viên có thể nắm được:

- Cách khai báo các kiểu dữ liệu mới để giải quyết theo yêu cầu của bài toán dựa vào những kiểu dữ liệu cơ bản được cài đặt sẵn trong ngôn ngữ lập trình.
- Biết nhập, xuất dữ liệu có cấu trúc và lưu trên mảng một chiều.

4.1.3 Chuẩn đầu ra

- Hiểu và định nghĩa được dữ liệu kiểu struct, khai báo và cách sử dụng biến kiểu struct.

4.2 NỘI DUNG

4.2.1 Tóm tắt kiến thức

Nắm được khái niệm cơ bản về kiểu dữ liệu có cấu trúc: là kiểu dữ liệu do người dùng tự định nghĩa, là kiểu dữ liệu gồm nhiều thành phần, mỗi thành phần có thể có kiểu khác nhau.

a) Định nghĩa kiểu dữ liệu cấu trúc:

Cách 1:

```
struct Tên_cấu_trúc
{
    Khai báo các biến thành phần;
    ....
};
typedef struct Tên_cấu_trúc Tên_kiểu;
```

Cách 2:

```
typedef struct Tên_cấu_trúc
{
    Khai báo các biến thành phần;
    ....
} Tên_kiểu;
```

b) Khai báo biến có kiểu cấu trúc để lưu dữ liệu:

```
Tên_kiểu Tên_biến;
```

c) Truy xuất từng thành phần của biến cấu trúc:

```
Tên_biến.Thành_phần;
```

d) Mảng cấu trúc

Cách khai báo tương tự như mảng một chiều (Kiểu dữ liệu bây giờ là kiểu dữ liệu có cấu trúc).

Cách truy cập phần tử trong mảng cũng như truy cập trên mảng một chiều. Nhưng do từng phần tử có kiểu cấu trúc nên phải chỉ định rõ cần lấy thành phần nào.

Nguyên tắc viết chương trình có mảng cấu trúc:

Do kiểu dữ liệu có cấu trúc thường chứa rất nhiều thành phần nên khi viết chương trình loại này ta cần lưu ý:

- Xây dựng hàm xử lý cho một cấu trúc.

- Muốn xử lý cho mảng cấu trúc, ta gọi lại hàm xử lý cho một cấu trúc đã được xây dựng bằng cách dùng vòng lặp.

4.2.2 Thực hành cơ bản

Bài 1: Làm quen với kiểu dữ liệu struct, xử lý 1 phần tử có kiểu struct.

Định nghĩa kiểu dữ liệu phân số gồm tử số và mẫu số. Viết chương trình thực hiện:

1. Nhập một phân số.
2. Xuất một phân số.
3. Rút gọn phân số.
4. Tính tổng, hiệu, tích và thương của hai phân số.
5. So sánh hai phân số.

Hướng dẫn:

- Câu 1, 2:

```
//Định nghĩa kiểu dữ liệu Phân số
typedef struct PhanSo
{
    int tu;
    int mau;
}PS;
//Nhập 1 phân số
void Nhap1PS(PS &x)
{
    //Nhập tử số
    printf("Nhập tu so: ");
    scanf("%d", &x.tu);
    //Nhập mẫu số khác 0, nếu nhập sai cho người dùng nhập lại
    do{
        printf("Nhập mau so khác 0: ");
        scanf("%d",&x.mau);
    }while(x.mau ==0);
    //Chuẩn hóa phân số
    if (x.mau<0) {
        x.tu=-x.tu;
        x.mau=-x.mau;
    }
}
```

```

}
//Xuất 1 phân số, xuất ra dưới dạng tu/mau, ví dụ: 3/5
void Xuat1PS(PS x)
{
    printf("%d/%d",x.tu,x.mau);
}

```

- Câu 3: để rút gọn phân số cần tìm giá trị ước chung lớn nhất của tử số và mẫu số. Do đó, ta định nghĩa 1 hàm để tìm giá trị ước chung lớn nhất của 2 số nguyên, sau đó vận dụng hàm này để rút gọn tử và mẫu của phân số.

```

//Tìm ước chung lớn nhất của 2 số nguyên, hàm trả về giá trị UCLN tìm được
int UCLN(int a, int b){
    a=abs(a); b=abs(b);
    Lặp chừng nào a≠b:
        Nếu a>b thì a=a-b;
        Ngược lại b=b-a;
    Trả về a; //hoặc trả về b, vì vòng lặp dừng khi a=b
}
//Rút gọn phân số
void rutGon(PS &x)
{
    if (x.tu==0) return; //nếu phân số có tử = 0 thì không cần rút gọn
    int uc=UCLN(x.tu, x.mau);
    x.tu=.....; //tính lại x.tu và x.mau
    x.mau=.....;
}

```

- Câu 4: tính tổng,hiệu, tích và thương của hai phân số, các mẫu hàm như sau:

```

PS tong2PS(PS x, PS y){
    PS tong; //phân số lưu kết quả sau khi cộng
    tong.tu= ....?; tong.mau=..?;
    RutGon(tong);
    return tong;
}

```

//Gọi thực hiện trong hàm main():

PS s=tong2PS(x,y);

printf("Tong 2 phan so la:"); xuat1PS(s);

//Các hàm còn lại tương tự:

PS hieu2PS(PS x, PS y);

PS tinh2PS(PS x, PS y);

PS thuong2PS(PS x, PS y);

- Câu 5: so sánh hai phân số. Hàm trả về:
 - + 0 nếu 2 phân số bằng nhau
 - + 1 nếu phân số thứ nhất > phân số thứ hai (theo thứ tự truyền tham số)
 - + -1 nếu phân số thứ nhất < phân số thứ hai (theo thứ tự truyền tham số)

```
int soSanh2PS(PS x, PS y)
```

```
{
```

```
    return x.tu*y.mau-y.tu*x.mau;
```

```
}
```

Bài 2: Mảng các phần tử có kiểu struct.

Kế thừa bài 1, viết chương trình thực hiện:

1. Nhập vào một dãy gồm n phân số ($0 < n < 100$)
2. Xuất dãy phân số.
3. Tính tổng các phân số trong dãy.
4. Tìm phân số lớn/nhỏ nhất.
5. Đếm số lượng phân số âm có trong dãy.
6. Sắp xếp dãy phân số tăng/giảm dần.

Hướng dẫn:

Tiếp theo bài 1, cài đặt các chức năng và chỉnh sửa hàm main().

- Lưu trữ số lượng phân số: `int n`;
- Lưu trữ dãy phân số: `PS a[MAX]`;

Các mẫu hàm như sau:

- + `void nhapSL(int &n);`

- + void nhapDay(PS a[], int n): vận dụng gọi hàm ***nhap1PS(a[i])***;
- + PS tongDay(PS a[], int n); vận dụng gọi hàm ***tong2PS(s, a[i])***; với PS s ban đầu có $s.tu=0$ và $s.mau=1$;
- + PS timMax(PS a[], int n); vận dụng gọi hàm ***soSanh2PS(a[i], max)***
- + int demAm(PS a[], int n);
- + void sapXep(PS a[], int n);

4.3 TỔNG KẾT

Sau bài thực hành này, người học cần hiểu và nắm vững:

- Nắm được các khái niệm cơ bản về kiểu dữ liệu có cấu trúc;
- Biết nhập, xuất dữ liệu có cấu trúc cho một phần tử;
- Biết nhập, xuất dữ liệu có cấu trúc và lưu trên mảng một chiều;

Bài thực hành 5: KIỂU DỮ LIỆU CÓ CẤU TRÚC (tiếp theo)

5.1 TỔNG QUAN

5.1.1 Giới thiệu

- Bài thực hành số 5: luyện tập kiểu dữ liệu cấu trúc gồm nhiều thành phần thông tin có kiểu khác nhau.
- Thời gian thực hành: trên lớp 3 tiết, tự học 3 tiết.
- Yêu cầu chuẩn bị: sinh viên đã nắm lý thuyết về kiểu dữ liệu cấu trúc.

5.1.2 Mục tiêu

Sau khi thực hành xong bài này, sinh viên có thể nắm được:

- Cách nhập, xuất một phần tử có kiểu cấu trúc;
- Cách nhập, xuất một danh sách (mảng 1 chiều) gồm nhiều phần tử, mỗi phần tử có kiểu cấu trúc gồm nhiều thành phần thông tin có kiểu khác nhau;
- Vận dụng các dạng thao tác xuất có điều kiện, đếm có điều kiện trên danh sách;
- Đi sâu vào các giải thuật trên mảng một chiều như tìm kiếm, sắp xếp, thêm phần tử, xóa phần tử theo từng thành phần của kiểu dữ liệu có cấu trúc.

5.1.3 Chuẩn đầu ra

- Vận dụng được cấu trúc dữ liệu mảng kiểu struct trong lưu trữ và xử lý dữ liệu cho các bài toán thực tế.

5.2 NỘI DUNG

5.2.1 Tóm tắt kiến thức

a) Mảng cấu trúc

Cách khai báo tương tự như mảng một chiều, kiểu của biến mảng là kiểu struct.

Cách truy cập phần tử trong mảng cũng như truy cập trên mảng một chiều. Nhưng do từng phần tử có kiểu cấu trúc nên phải chỉ định rõ cần lấy thành phần nào.

b) Nguyên tắc viết chương trình có mảng cấu trúc:

Do kiểu dữ liệu có cấu trúc thường chứa rất nhiều thành phần nên khi viết chương trình loại này ta cần lưu ý:

- Xây dựng hàm xử lý cho một cấu trúc.
- Muốn xử lý cho mảng cấu trúc, ta gọi lại hàm xử lý cho một cấu trúc đã được xây dựng bằng cách dùng vòng lặp.

5.2.2 Thực hành cơ bản

Bài 1: Viết chương trình quản lý danh sách sinh viên, trong đó thông tin của mỗi sinh viên gồm:

- o mã sinh viên
- o họ tên
- o giới tính (x: nữ, y: nam)
- o ngày sinh (gồm ngày, tháng và năm)
- o lớp (chuỗi 8 ký tự với 2 ký tự đầu là năm học, 1 ký tự tiếp theo là bậc học (D: đại học, C: cao đẳng), 3 ký tự tiếp theo là ngành học, 2 ký tự cuối là số thứ tự của lớp. Ví dụ: 22DITE01.
- o điểm trung bình.

Chương trình có các chức năng sau:

1. Nhập danh sách sinh viên.
2. Xuất danh sách sinh viên.
3. Xuất các sinh viên có điểm trung bình >5.
4. Xuất các sinh viên có sinh nhật trong tháng 12.
5. Xuất danh sách sinh viên thuộc ngành công nghệ thông tin
6. Xuất các sinh viên có điểm trung bình cao nhất.
7. Đếm số lượng sinh viên nữ.
8. Thêm một sinh viên mới vào cuối danh sách.
9. Tìm sinh viên có mã là X, nếu tìm thấy hãy cập nhật lại tên lớp của sinh viên đó.
10. Xoá sinh viên có mã là X khỏi danh sách.
11. *Sắp xếp danh sách tăng theo điểm trung bình.

Hướng dẫn:

Trước hết ta phải định nghĩa kiểu dữ liệu ngày tháng năm (đặt tên kiểu là Date), định nghĩa kiểu dữ liệu sinh viên (đặt tên kiểu là SV).

Xây dựng hàm nhập và xuất cho một ngày tháng năm.

Muốn nhập được danh sách sinh viên ta phải xây dựng hàm nhập cho 1 sinh viên, sau đó xây dựng hàm nhập cho danh sách sinh viên. Tham khảo đoạn code sau minh họa cho câu 1, 2:

```
#define MAX 100
//định nghĩa kiểu dữ liệu ngày-tháng-năm
typedef struct date
{
    unsigned char ngay;
    unsigned char thang;
    int nam;
}Date;
//định nghĩa kiểu dữ liệu sinh viên
typedef struct SinhVien
{
    char ma[11];
    char hoten[31];
    Date ns; //ngày sinh
    char gt; //giới tính lưu đại diện bởi 1 ký tự x: nữ, y: nam
            //HOẶC int gt; //1: nam, 0: nữ
    char lop[9];
    float dtb;
}SV;
//khai báo các nguyên mẫu hàm sẽ dùng trong chương trình
void nhapNS(Date &d);
void xuatNS(Date d);
void nhap1SV (SV &x);
void xuat1SV (SV x);
void nhapSL(int &n);
void nhapDS(SV a[], int n);
void xuatDS(SV a[], int n);
//=====
//hàm chính của chương trình
int main()
{
```

```

SV a[MAX]; //Khai báo mảng a gồm có tối đa MAX sinh viên
int n; //n lưu số lượng sinh viên của danh sách
nhapSL(n); //nhập số lượng sinh viên
nhapDS(a, n); //nhập danh sách
xuatDS(a, n); //xuất danh sách
return 0;
}
//-----
//Code chi tiết các hàm con
void nhapNS(Date &d)
{
    printf("\nNhập vào ngày: ");
    scanf("%u", &d.ngay);
    printf("\nNhập vào tháng: ");
    scanf("%u", &d.thang);
    printf("\nNhập vào năm: ");
    scanf("%d", &d.nam);
}
void xuatNS(Date d)
{
    printf("%02u/%02u/ %4d", d.ngay, d.thang, d.nam);
}
//-----
//nhập thông tin cho 1 sinh viên
void nhap1SV (SV &x){
    printf("Nhập mã sinh viên: "); scanf("%s", &x.ma);

    printf("Nhập họ tên: ");
    fflush(stdin); //Xóa vùng đệm
    gets(x.hoten);

    printf("Nhập ngày tháng năm sinh: "); nhapNS(x.ns);
    printf("Nhập lop: "); scanf("%s", &x.lop);

    do{
        printf("Nhập giới tính x-nu, y-nam: ");
        x.gt=getche();
    }while (x.gt!='x' && x.gt!='y'); //hằng kí tự cụ thể phải đặt trong '..'

```

```

printf("\nNhap vao diem trung binh: "); scanf("%f", &x.dtb);
}
//-----
//xuất thông tin của 1 sinh viên ra màn hình
void xuat1SV (SV x)
{
    printf("%-11s\t-30s\t", x.ma, x.hoten);
    if (x.gt=='x') printf("nu\t"); else printf("nam\t");
    xuatNgS(x.ns);
    printf("\t%-8s\t%.1f\n", x.lop, x.dtb);
} //nếu xuất ra chưa đẹp, hãy chỉnh sửa giá trị thông số trong chuỗi định dạng ☺
//-----
//Nhập số lượng sinh viên của danh sách
void nhapSL(int &n)
{
    //nhập số lượng n>0, nếu nhập sai thì cho nhập lại
}
//Nhập thông tin của từng sinh viên trong danh sách
void nhapDS(SV a[], int n)
{
    printf("\nNHAP DANH SACH SINH VIEN----- \n:");
    for(int i=0; i<n; i++)
    {
        printf("\nNhap sinh vien thu %d:\n", i+1);
        nhap1sv(a[i]); //Gọi hàm nhập thông tin cho 1 sinh viên thứ i trong ds.
    }
}
//-----
//xuất danh sách sinh viên ra màn hình
void xuatDS(SV a[], int n)
{
    printf("\nDANH SACH SINH VIEN-----\n");
    for(int i=0; i<n; i++)
        xuat1SV(a[i]); // Gọi hàm xuất thông tin cho 1 sinh viên thứ i trong ds.
}

```

- Câu 3, 4, 5, 6: dạng hàm xuất có điều kiện

```

void xuấtTheoDK(Kiểu a[], int n)
{
    printf("Cac phan tu thoa dieu kien:\n"); //sửa câu thông báo cho phù hợp
    for(int i=0; i<n; i++)
        if (điều kiện trên a[i])
            xuất a[i];
}
//Gọi thực hiện trong main():    xuấtTheoDK(a, n);

```

Câu 6: viết hàm tìm giá trị điểm trung bình cao nhất (tìm max điểm), sau đó viết hàm xuất các sinh viên có điểm trung bình = điểm tb cao nhất.

Hàm tìm điểm cao nhất: float maxDTB(SV a[], int n);

Vận dụng:

- float max=maxDTB(a,n);
 - điều kiện xuất: if (a[i].dtb==max)
- Câu 7: đếm số lượng sinh viên nữ, áp dụng dạng hàm đếm có điều kiện.
- Câu 8: thêm một sinh viên mới vào cuối danh sách, áp dụng hàm thêm 1 phần tử tại vị trí k.

void themTaiViTri(SV a[], int &n, int k, SV x);

Gọi thực hiện trong main(), thực hiện các bước sau:

- + Nhập sinh viên cần thêm, gọi hàm **nhap1SV(x)**;
 - + Gọi hàm thêm x vào vị trí cuối danh sách tức $k=n$: **themTaiViTri(a, n, n, x)**;
 - + Gọi hàm xuất danh sách để xem danh sách sau khi thêm.
- Câu 9: tìm sinh viên có mã là X, nếu tìm thấy hãy cập nhật lại tên lớp của sinh viên đó.
- + Tìm sinh viên có mã X, nếu tìm thấy trả về vị trí của SV đó trong danh sách, nếu không thì trả về -1: int timSV(SV a[], int n, char X[]);
 - + Gọi hàm tìm và thực hiện sửa tên lớp:

```

int vt=timSV(a,n,X);
if (vt==-1) printf("\nKhong tim thay");
else{
    //nhập tên lớp mới

```

```
//sửa tên lớp cho người tìm được trong danh sách  
}
```

- Câu 10: xoá sinh viên có mã là X khỏi danh sách, sử dụng 2 hàm sau:
 - + int timSV(SV a[], int n, char X[]);
 - + xoaTaiViTri(SV a[], int &n, int k);

5.3 TỔNG KẾT

Sau bài thực hành này, người học cần hiểu và nắm vững:

- Cách định nghĩa kiểu dữ liệu có cấu trúc;
- Biết nhập, xuất dữ liệu có cấu trúc cho một phần tử;
- Biết nhập, xuất dữ liệu có cấu trúc và lưu trên mảng một chiều;
- Đi sâu vào các giải thuật trên mảng một chiều như tìm kiếm, sắp xếp, thêm phần tử, xóa phần tử theo từng thành phần của kiểu dữ liệu có cấu trúc.

Bài thực hành 6: KIỂU DỮ LIỆU CÓ CẤU TRÚC (tiếp theo)

6.1 TỔNG QUAN

6.1.1 Giới thiệu

- Tiếp tục luyện tập kiểu dữ liệu cấu trúc gồm nhiều thành phần thông tin có kiểu khác nhau.
- Thời gian thực hành: trên lớp 3 tiết, tự học 3 tiết.
- Yêu cầu chuẩn bị: sinh viên đã nắm lý thuyết về kiểu dữ liệu cấu trúc. .

6.1.2 Mục tiêu

Sau khi thực hành xong bài này, sinh viên có thể nắm được:

- Vận dụng các dạng thao tác tính tổng/đếm/trung bình cộng... trên danh sách;
- Cài đặt được các thao tác trên dữ liệu danh sách ứng dụng thực tế.

6.1.3 Chuẩn đầu ra

- Vận dụng được cấu trúc dữ liệu mảng kiểu struct trong lưu trữ và xử lý dữ liệu cho các bài toán thực tế.

6.2 NỘI DUNG

6.2.1 Thực hành cơ bản

Viết chương trình quản lý sách cho một cửa hàng sách, sử dụng mảng 1 chiều để lưu các cuốn sách; thông tin của mỗi đầu sách gồm: mã sách, tên sách, tên tác giả, loại sách (gồm 2 loại Tự nhiên và Xã hội), năm xuất bản, giá tiền, số lượng.

Chương trình có các chức năng sau:

1. Nhập thông tin các cuốn sách.
2. Xuất thông tin các cuốn sách.
3. Tính tổng thành tiền tất cả các cuốn sách.

4. Xuất các cuốn sách có năm xuất bản trước năm 2000.
5. Đếm số lượng sách có giá lớn hơn 50000.
6. Xuất các cuốn sách theo loại (xuất loại nào là do người dùng chọn).

Hướng dẫn:

Định nghĩa kiểu dữ liệu để lưu thông tin của 1 đầu sách.

Cài đặt các hàm với nguyên mẫu hàm như sau:

```
void nhap1Sach(Sach &x); //khi nhập loại sách yêu cầu người dùng chỉ nhập 1  
trong hai loại 1: tự nhiên, 2: xã hội; nếu nhập sai thì phải nhập lại  
void xuat1Sach(Sach x);  
void nhapSL(int &n);  
void nhapDS(Sach a[], int n);  
void xuatDS(Sach a[], int n);  
long thanhTien(Sach a[], int n);  
void xuatTheoNam(Sach a[], int n);  
int demTheoGia(Sach a[], int n);  
void xuatTheoLoai(Sach a[], int n, int loai);
```

6.3 TỔNG KẾT

Sau bài thực hành này, người học cần hiểu và nắm vững:

- Kiểu dữ liệu có cấu trúc cho phép ta định nghĩa những kiểu dữ liệu bất kỳ trên cơ sở là những kiểu dữ liệu cơ bản có sẵn trong ngôn ngữ lập trình.
- Khi xây dựng xong kiểu dữ liệu mới ta phải định nghĩa những thao tác cho kiểu dữ liệu đó.
- Những kiểu dữ liệu tự định nghĩa này thông thường có rất nhiều thành phần, mỗi thành phần cũng có thể là một kiểu dữ liệu tự định nghĩa, vấn đề là ta chọn kiểu dữ liệu cơ bản nào để xây dựng nên chúng sao cho phù hợp về mặt kiểu dữ liệu và phù hợp về kích thước lưu trữ (vừa đủ).
- Các sử dụng những kiểu dữ liệu tự định nghĩa cũng giống như các kiểu dữ liệu cơ bản. Muốn sử dụng phải khai báo biến, khi truy cập các thành phần phải truy cập theo quy ước.

- Đối với mảng các kiểu dữ liệu có cấu trúc ta nên xử lý cho từng thành phần cấu trúc rồi mới xử lý cho mảng cấu trúc bằng cách dùng vòng lặp.

Bài thực hành 7: KIỂU CON TRỎ

7.1 TỔNG QUAN

7.1.1 Giới thiệu

- Bài thực hành số 7: luyện tập sử dụng biến động – biến con trỏ.
- Thời gian thực hành: trên lớp 3 tiết, tự học 3 tiết.
- Yêu cầu chuẩn bị: sinh viên đã nắm lý thuyết về biến động kiểu con trỏ.

7.1.2 Mục tiêu

Sau khi thực hành xong bài này, sinh viên có thể nắm được:

- Cách khai báo và sử dụng biến kiểu con trỏ;
- Biết xử lý các phép toán trên mảng một chiều theo kiểu con trỏ;
- Con trỏ với kiểu dữ liệu có cấu trúc.

7.1.3 Chuẩn đầu ra

- Hiểu và sử dụng được biến động.
- Cài đặt được một số bài toán cơ bản bằng con trỏ.

7.2 NỘI DUNG

7.2.1 Tóm tắt kiến thức

a) Định nghĩa

- Biến con trỏ không chứa dữ liệu mà chỉ chứa địa chỉ của ô nhớ chứa dữ liệu.
- Kích thước của biến con trỏ không phụ thuộc vào kiểu dữ liệu, luôn có kích thước cố định là 2 byte.

b) Khai báo biến con trỏ

Cú pháp: Kiểu *Tên con trỏ;

c) Cấp phát bộ nhớ cho biến con trỏ quản lý

biến=(kiểu*) malloc(size tổng cần cấp phát);

biến=(kiểu*) calloc(số lượng phần tử, size của mỗi phần tử);

```
biến=realloc(biến trỏ đến vùng nhớ đã được cấp phát trước đó, size mới);
```

- Để thu hồi bộ nhớ ta có thể dùng hàm **free()**.
- *Lưu ý: Trong C++ có thể cấp phát bộ nhớ bằng toán tử **new** và thu hồi bằng toán tử **delete**.*

d) So sánh cấp phát bộ nhớ tĩnh và động

Cấp phát bộ nhớ tĩnh – dùng biến tĩnh	Cấp phát bộ nhớ động – dùng biến con trỏ
Bộ nhớ được cấp phát trước khi chạy chương trình (trong quá trình biên dịch).	Bộ nhớ được cấp phát trong quá trình chạy chương trình.
Không thể cấp phát hay phân bổ lại bộ nhớ trong khi chạy chương trình.	Cho phép quản lý, phân bổ hay giải phóng bộ nhớ trong khi chạy chương trình.
Vùng nhớ được cấp phát và tồn tại cho đến khi kết thúc chương trình.	Chỉ cấp phát vùng nhớ khi cần sử dụng tới.
Chương trình chạy nhanh hơn so với cấp phát động.	Chương trình chạy chậm hơn so với cấp phát tĩnh.
Tốn nhiều không gian bộ nhớ hơn.	Tiết kiệm được không gian bộ nhớ sử dụng.

Ưu điểm chính của việc sử dụng cấp phát động là giúp ta tiết kiệm được không gian bộ nhớ mà chương trình sử dụng. Bởi vì chúng ta sẽ chỉ cấp phát khi cần dùng và có thể giải phóng vùng nhớ đó ngay sau khi sử dụng xong. Tuy nhiên, nhược điểm chính của cấp phát động là bạn phải tự quản lý vùng nhớ mà bạn cấp phát. Nếu bạn cứ cấp phát mà quên giải phóng bộ nhớ thì chương trình của bạn sẽ tiêu thụ hết tài nguyên của máy tính dẫn đến tình trạng tràn bộ nhớ (memory leak).

7.2.2 Thực hành cơ bản

Bài 1: Hãy chạy đoạn chương trình sau và rút ra nhận xét về biến con trỏ.

```
int main()
{
    //Khai bao bien n và con trỏ p;
```

```
int n, *p;

//lưu dữ liệu với biến tĩnh
printf("Nhap gia tri cho bien N:");
scanf("%d", &n);
printf("Gia tri bien N = %d",n);

//lưu dữ liệu với biến động
p = (int *) malloc(sizeof(int)); //Cap phat bo nho cho con tro p quản lý;
printf("\nNhap gia tri cho con tro p:");
scanf("%d", p); //Do p lưu địa chỉ của vùng nhớ chứa dữ liệu nên không cần &
printf("Gia tri con tro p= %d",*p);

*p = *p +5; //Tang gia tri cua p len 5;
printf("\nGia tri p sau khi tang: %d",*px);
free(p);
return 0;
}
```

Bài 2: Dùng phương pháp con trỏ, viết chương trình thực hiện:

1. Nhập dãy số nguyên gồm n phần tử, trong đó:
 - a. Biến lưu số lượng phần tử trong dãy: n là biến bình thường
 - b. Biến lưu dãy số: dùng biến con trỏ: `int *a;`
2. Xuất mảng.
3. Xuất ra màn hình các phần tử là số nguyên tố nằm tại những vị trí lẻ trong mảng.
4. Xuất ra vị trí của các phần tử có giá trị lớn nhất.
5. Tìm phần tử có giá trị X.
6. Tính tổng các phần tử nằm ở vị trí chẵn trong mảng.

Bài 3: Dùng phương pháp con trỏ để lưu danh sách:

Viết chương trình quản lý vé xem phim, thông tin một vé xem phim như sau :

- Mã ghế
- Tên phim

- Loại phim (1: kinh dị, 2: hành động, 3: tình cảm).
- Mã phòng chiếu
- Giá tiền

Thực hiện các yêu cầu sau:

Khai báo cấu trúc dữ liệu để lưu thông tin của 1 vé xem phim.

Viết các hàm thực hiện chức năng nhập danh sách các vé xem phim.

1. Nhập số lượng vé trong danh sách
2. Nhập thông tin cho 1 vé
3. Nhập danh sách các vé
4. Xuất danh sách các vé xem phim theo loại (với loại tùy ý người dùng).
5. Đếm trong danh sách có bao nhiêu vé phim tình cảm.
6. Tìm vé có mã số ghế là X với X do người dùng nhập.
7. Xóa vé xem phim ở đầu danh sách .

*Hướng dẫn: biến mảng lưu danh sách vé xem phim dùng con trỏ: VE *a;*

7.3 TỔNG KẾT

Sau bài thực hành này, người học cần hiểu và nắm vững:

- Phân biệt ý nghĩa và cách sử dụng biến tĩnh và biến động, ưu nhược điểm của mỗi loại.
- Cách khai báo, sử dụng biến con trỏ, cấp phát và quản lý bộ nhớ bởi biến con trỏ.
- Áp dụng biến con trỏ cho mảng 1 chiều, mảng struct.

Bài thực hành 8: ĐỆ QUY

8.1 TỔNG QUAN

8.1.1 Giới thiệu

- Bài thực hành số 8: luyện tập kỹ thuật giải bài toán bằng đệ quy.
- Thời gian thực hành: trên lớp 3 tiết, tự học 3 tiết.
- Yêu cầu chuẩn bị: sinh viên đã nắm lý thuyết về đệ quy.

8.1.2 Mục tiêu

Sau khi thực hành xong bài này, sinh viên có thể nắm được:

- Phương pháp lập trình theo kỹ thuật đệ quy, cách hoạt động và cách cài đặt các hàm đệ quy.

8.1.3 Chuẩn đầu ra

- Hiểu và nhận biết được hàm đệ quy, cách thức hoạt động. Cài đặt được một số bài toán cơ bản bằng đệ quy.

8.2 NỘI DUNG

8.2.1 Tóm tắt kiến thức

a) Định nghĩa

Một hàm được gọi có tính đệ qui nếu trong thân của hàm đó có lệnh gọi lại chính nó.

b) Kỹ thuật giải bài toán bằng đệ quy:

1. Thông số hóa bài toán.
2. Tìm giải thuật tổng quát theo hướng đệ quy lui dần về tình huống bị chặn.
3. Tìm các điều kiện biên (chặn, dừng), tìm giải thuật cho các tình huống này.

Lưu ý:

- Đệ qui cung cấp cho ta cơ chế giải quyết các bài toán phức tạp một cách đơn giản hơn.
- Xây dựng hàm đệ qui thông qua việc xác định điều kiện dừng và bước thực hiện tiếp theo.

- Chỉ nên cài đặt bằng phương pháp đệ quy khi không còn cách giải quyết bằng cách lặp thông thường.

8.2.2 Thực hành cơ bản

Viết chương trình thực hiện (dùng đệ quy):

1. Nhập 1 mảng số nguyên.
2. Xuất mảng số nguyên
3. Tính tổng các phần tử của mảng
4. Tính tổng các phần tử chẵn của mảng
5. Đếm số lượng phần tử dương
6. Tìm phần tử lớn nhất (nhỏ nhất) của mảng
7. Tìm phần tử chẵn cuối cùng có trong mảng
8. Nhập 1 trị x, tìm vị trí có x cuối cùng trong mảng.

Hướng dẫn:

```
void Nhap(int a[], int n)
{
    if(n==0)
        return;
    Nhap(a, n-1); //nhập cho n-1 phần tử đầu
    printf("\nNhập phần tử thu %d: ", n-1);
    scanf("%d", &a[n-1]); //nhập cho phần tử cuối trong mảng
}

void Xuat(int a[], int n)
{
    if(n==0)
        return;
    Xuat(a, n-1);
    printf("%d\t", a[n-1]);
}
```

8.3 TỔNG KẾT

Sau bài thực hành này, người học cần hiểu và nắm vững:

- Đệ quy cung cấp cho ta cơ chế giải quyết các bài toán phức tạp một cách đơn giản hơn.
- Xây dựng hàm đệ quy thông qua việc xác định điều kiện dừng và bước thực hiện tiếp theo.
- Chỉ nên cài đặt bằng phương pháp đệ quy khi không còn cách giải quyết bằng cách lặp thông thường.

Bài thực hành 9: TẬP TIN

9.1 TỔNG QUAN

9.1.1 Giới thiệu

- Bài thực hành số 9: sử dụng tập tin để lưu dữ liệu và đọc dữ liệu ra khi cần.
- Thời gian thực hành: trên lớp 3 tiết, tự học 3 tiết.
- Yêu cầu chuẩn bị: sinh viên đã nắm lý thuyết về kiểu dữ liệu tập tin, các hàm thư viện để thao tác với tập tin.

9.1.2 Mục tiêu

Sau khi thực hành xong bài này, sinh viên có thể nắm được:

- Cấu trúc tập tin, cài đặt các thao tác: mở file, đọc/ghi file với các kiểu dữ liệu, đóng file;
- Một số hàm thư viện và ứng dụng trong việc tổ chức dữ liệu trên tập tin.

9.1.3 Chuẩn đầu ra

- Sử dụng được kiểu dữ liệu tập tin để lưu trữ dữ liệu và các thao tác cơ bản trên tập tin.

9.2 NỘI DUNG

9.2.1 Tóm tắt kiến thức

Trong các chương trình trước thì các dữ liệu đưa vào chương trình chỉ được tồn tại trong RAM, khi thoát chương trình thì tất cả dữ liệu đều bị mất. Để khắc phục tình trạng này Dev-C cung cấp cho ta các hàm để lưu trữ và truy xuất tập tin, đó là kiểu FILE. Và ở đây ta chỉ đề cập đến 2 loại tập tin:

- Tập tin văn bản: là tập tin dùng để ghi các ký tự lên đĩa theo các dòng.
- Tập tin nhị phân: là tập tin dùng để ghi các cấu trúc dạng nhị phân (được mã hoá).

Quá trình thao tác trên tập tin thông qua 4 bước:

- Bước 1: Khai báo con trỏ trỏ đến tập tin.
- Bước 2: Mở tập tin.

- Bước 3: Các xử lý trên tập tin.
- Bước 4: Đóng tập tin.

Khai báo biến để truy cập tập tin:

FILE * tên biến;

Ví dụ : FILE *f;

Mở tập tin:

tên biến file = fopen(đường dẫn đến tập tin, “kiểu truy cập”);

Ví dụ : FILE *f = fopen (“C:\\VD1.txt” , “rt”) ;

Các kiểu truy nhập tập tin thông dụng:

t là kiểu truy nhập tập tin đối với dạng tập tin văn bản (text).

b là kiểu truy nhập tập tin đối với dạng tập tin nhị phân (binary).

r mở ra để đọc (ready only).

w mở ra để ghi mới (create / write).

a mở ra để thêm vào (append).

r+ mở ra để đọc và ghi (modify).

Đóng tập tin:

fclose(biến con trỏ tập tin) ;
hoặc fcloseall() ;

Các hàm đọc ghi tập tin văn bản:

TÊN HÀM	Ý NGHĨA	VÍ DỤ
Đọc dữ liệu từ tập tin ra biến (nhập dữ liệu từ file lưu vào biến)		
fscanf(biến file, “định dạng”, các tham biến);	Đọc dữ liệu từ một tập tin theo định dạng.	int x; float y; fscanf(f, “%d%f”, &x, &y);
fgets(biến chuỗi, kích thước tối đa, biến file);	Đọc một chuỗi ký tự từ một tập tin với kích thước tối đa cho phép, hoặc gặp ký tự xuống dòng.	char s[80]; fgets(s, 79, f);

biến kí tự = getc(biến file);	Đọc một ký tự từ tập tin đang mở.	char ch=getc(f);
Ghi tập tin (xuất dữ liệu (từ biến) ra tập tin)		
fprintf(biến file, “chuỗi định dạng” [, <các tham biến nếu có>]);	Ghi dữ liệu theo một định dạng nào đó vào tập tin.	fprintf(f, “Ket qua la %d\n”, x);
fputs(chuỗi ký tự, biến file);	Ghi một chuỗi ký tự vào tập tin đang mở.	fputs(“Chuong trinh minh hoa”, f);

Các hàm đọc ghi tập tin nhị phân

TÊN HÀM	Ý NGHĨA	VÍ DỤ
Đọc dữ liệu từ tập tin ra biến (nhập dữ liệu cho biến từ file)		
fread(&ptr, size, len, biến file);	- ptr: vùng nhớ để lưu dữ liệu đọc. - size: kích thước mỗi ô nhớ (tính bằng byte). - len: độ dài dữ liệu cần đọc.	int a[30], n; fread(a, sizeof(int), n, f); SV b; Fread(&b, sizeof(SV), 1, f);
Ghi tập tin (ghi dữ liệu (từ biến) ra tập tin)		
fwrite(&p, size, len, biến file);	Tham số tương tự như hàm fread.	fwrite(a, sizeof(int), n, f);

9.2.2 Thực hành cơ bản

Bài 1: Cho file TXT có cấu trúc sau:

- Dòng đầu lưu giá trị một số nguyên dương n
- Dòng còn lại lưu giá trị của một dãy gồm n các số nguyên.
 1. Đọc file trên, lưu dữ liệu đọc được vào mảng một chiều.
 2. Xuất mảng ra màn hình
 3. Ghi các số nguyên tố có trong mảng vào file (ghi tiếp theo vào file đã có).

Hướng dẫn:

Tạo file notepad chứa dữ liệu như sau (bạn có thể tạo dữ liệu khác):

```
10
1 3 -7 19 92 -25 5 90 17 23
```

```
#include<stdlib.h>
#define MAX 100

//đọc/nhập dữ liệu từ file
void docFile(char duongdan[], int a[], int &n)
{
    //mở file theo đường dẫn đã cho
    FILE *f = fopen(duongdan, "r");
    if(f==NULL)
    {
        printf("\nKhong mo duoc file.");
        return;
    }
    else
    {
        //đọc số lượng phần tử lưu vào biến n
        fscanf(.....);
        //đọc lần lượt n số lưu vào mảng
        for(.....)
            fscanf(f, "%d", &a[i]);
        fclose(f);
    }
}

//xuất mảng ra màn hình
void xuatMAng(int a[], int n){...}
//ghi các số nguyên tố vào file
void ghiFile(char duongdan[], int a[], int n){...}
//=====
int main(){
    int a [MAX], n;
    char duongdan[30];
    printf("Nhap duong dan den file: "); gets(duongdan);
    docFile(duongdan, a, n);
    xuatMang(a,n);
    ghiFile(a,n);
    return 0;
}
```

Bài 2: Viết chương trình phát sinh ngẫu nhiên ma trận a kích thước 5×6 , lưu ma trận này vào file **test.inp**. Đọc lại file test.inp đưa dữ liệu vào ma trận b và xuất ra màn hình xem kết quả lưu đúng không? Cấu trúc của file test.inp như sau:

- Dòng đầu lưu 2 số nguyên: d, c thể hiện số dòng và số cột của ma trận.
- d dòng tiếp theo, mỗi dòng gồm c phần tử là giá trị các phần tử trên một dòng của ma trận.

Hướng dẫn:

```
#include<stdlib.h>
#define MAX 100

//sinh ma trận ngẫu nhiên
void sinhMT(int a[][MAX], int d, int c){...}
//ghi ma trận vào file
void ghiFile(char duongdan[], int a[][MAX], int d, int c)
{
    FILE *f;
    f=fopen(duongdan, "wt");
    if(f==NULL)
        //....
    fprintf(f, "%d %d\n", d, c);
    for(int i=0; i<d; i++)
    {
        for(int j=0; j<c; j++)
            fprintf(f, "%d\t", a[i][j]);
        fprintf(f, "\n");
    }
    fclose(f);
}

//Đọc dữ liệu từ file ra biến mảng lưu ma trận-----
void DocFile(char duongdan[], int a[][MAX], int &d, int &c)
{
    + mở file như trên
    + đọc số dòng, số cột
```

```
+   đọc giá trị các phần tử của ma trận  
+   đóng file  
}
```

Bài 3: Viết chương trình cho phép nhập từ bàn phím và ghi vào 1 tập tin tên DSHH.TXT với mỗi phần tử của tập tin là một cấu trúc bao gồm các trường:

- mh (mã hàng: char[5]).
- sl (số lượng: int).
- dg (đơn giá: float).
- st (Số tiền: float) = số lượng * đơn giá.

Sau khi nhập xong yêu cầu in toàn bộ danh sách hàng hóa ra màn hình.

9.3 TỔNG KẾT

Sau bài thực hành này, người học cần hiểu và nắm vững:

- Mục đích của kiểu dữ liệu tập tin cho phép chúng ta lưu lại những thông tin cần thiết tương đối lớn: những dữ liệu đầu vào, những kết quả của chương trình hoặc những dữ liệu dùng để test chương trình, ...
- Khi thao tác trên tập tin phải thông qua 4 bước: *Khai báo con trỏ trỏ đến tập tin, Mở tập tin, Xử lý trên tập tin và cuối cùng là Đóng tập tin.*
- Lưu ý khi mở tập tin để ghi thì phải cẩn thận với thao tác tạo mới hay chỉnh sửa nội dung tập tin, di chuyển con trỏ hợp lý để tránh mất thông tin.
- Sử dụng hàm thao tác trên tập tin phải dùng **đúng loại hàm** cho tập tin kiểu nhị phân hay kiểu văn bản.

